Verifying the uNPU from scratch

How twelve self-checking testbenches, a C golden model and deliberate bugs prove a 4×4 systolic accelerator correct

EE5239 · Project 2 · Frankenstein 180 nm IP-Validation SoC
Matches `tb/` (12 testbenches), `model/golden.c` and `scripts/run_xrun.sh`; results from `docs/freeze-report-v2.md`
November 2026

What is inside

1. What “verified” has to mean
2. A testbench from nothing
3. Where the right answer comes from: the golden model
4. Choosing inputs: five strategies
5. Growing the device under test, one block at a time
6. The compute core: PE, grid, skew, stall
7. The buffers: proving double-banking
8. Memory and buses: the SRAM model and back-pressure
9. Control: sequencer, registers, APB
10. The whole macro, and a second opinion
11. Running a regression
12. Testing the tests: mutation, second simulator, random init
13. What verification found
14. Limits, and where to go next
15. Appendix: commands, files, counts

1. What “verified” has to mean

A chip cannot be patched after fabrication. Every bug that reaches the GDS is permanent, so the RTL has to be shown correct **before** tape-out, and simulation is the main tool.

“Correct” is not one thing. For the uNPU it means four separate promises, each checked by different tests:

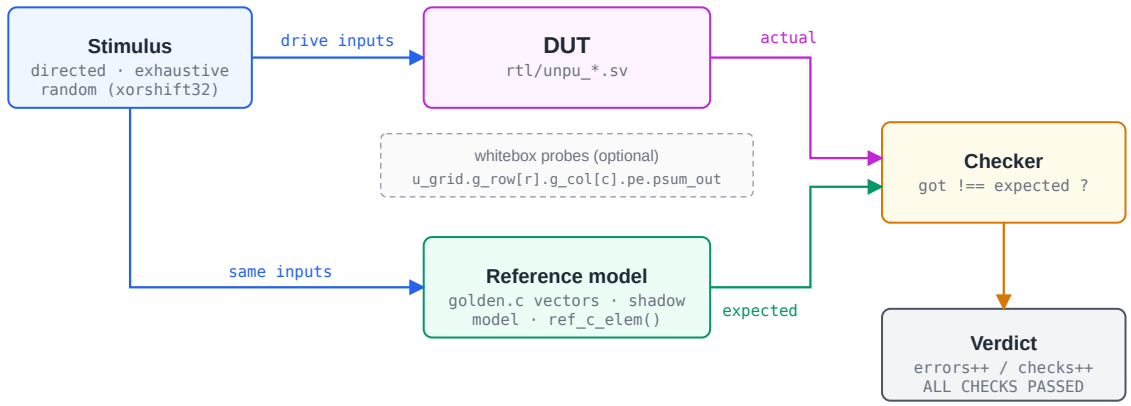
Promise	Example of breaking it	Checked by
Arithmetic is right	`0xFF × 0xFF` read as 65 025 in signed mode	`pe`, `grid`, every end-to-end TB
Timing contract holds	a row of results lands one cycle late	`grid`, `skew`, `stall`, `seq`
Protocols are obeyed	`dma_addr` changes while the memory is stalling	`apb`, `dma`, `top`, `ext2`
Control behaves	a second START during a job corrupts it	`seq`, `csr`, `top`, `ext2`

A test only proves something if it **can fail**. Much of this document is about making sure that it can.

2. A testbench from nothing

A testbench is a SystemVerilog module with no ports. It instantiates the design (the **DUT**, device under test), makes a clock, drives inputs, and checks outputs.

Anatomy of a self-checking testbench (every file in tb/ has this shape)



The checker compares with !== (4-state), so an X or Z from the DUT counts as a mismatch, never a silent pass.

The shape every testbench here follows

Here is a complete, minimal self-checking test for one processing element, written the same way as `tb/unpu_pe_tb.sv`:

```
        `timescale 1ns/1ps
        module tiny_pe_tb;
          logic clk = 0, rst_n, array_en, weight_load, mode_unsigned;
          logic [7:0]  weight_in, act_in, act_out;
          logic [31:0] psum_in, psum_out;
          int errors = 0;

          unpu_pe dut (.*);                    // the device under test
          always #5 clk = ~clk;                // 100 MHz in simulation; speed is irrelevant
here

          task automatic step;  @(posedge clk); #1;  endtask   // edge, then let NBAs settle

          initial begin
            rst_n = 0; array_en = 0; weight_load = 0; mode_unsigned = 0;
            weight_in = 0; act_in = 0; psum_in = 0;
            step(); rst_n = 1;

            array_en = 1; weight_load = 1; weight_in = 8'hFF; step();   // load weight = -1
            weight_load = 0; act_in = 8'hFF; psum_in = 32'd10; step();  // -1 * -1 + 10
            if (psum_out !== 32'd11) begin errors++; $display("FAIL got %0d", psum_out); end

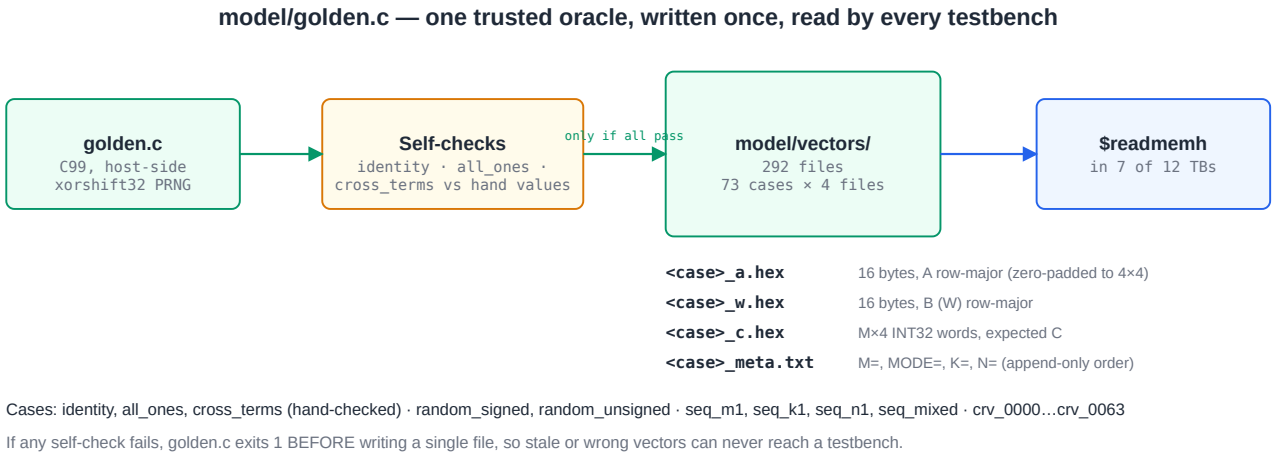
            if (errors == 0) $display("ALL CHECKS PASSED");
            $finish;
          end
        endmodule
```

Five habits from this tiny file run through all 10 550 lines of `tb/`:

1. **Self-checking.** The testbench prints PASS or FAIL itself. Nobody has to read waveforms to know the answer.
2. **`step()` waits 1 ns after the edge.** Non-blocking assignments (`<=`) update at the edge, so checking a hair later always sees the new values.
3. **`!==` not `!=`.** The 4-state operator treats `X` and `Z` as mismatches. With `!=`, an unknown value can slip through as "not unequal".
4. **Count everything.** `errors` and `checks` counters, so a run reports *how much* it verified.
5. **One unambiguous final line**, which scripts can search for.

3. Where the right answer comes from: the golden model

A check needs an expected value. Hand-computing them works for five cases, not for five thousand. `model/golden.c` is a small C program that computes `C = A × W` exactly as the hardware should, and writes the answers to files every testbench can read.



golden.c: compute, self-check, then write

3.1 One MAC, two modes

The PE multiplies 8-bit values either as signed (−128…127) or unsigned (0…255). The model mirrors that with C's own type conversions:

```
        static int32_t mac_product(uint8_t a, uint8_t w, npu_mode_t mode) {
          if (mode == MODE_UNSIGNED) return (int32_t)((uint32_t)a * (uint32_t)w);
          int8_t as = (int8_t)a, ws = (int8_t)w;            /* reinterpret as two's complement
*/
          return (int32_t)as * (int32_t)ws;
        }
```

3.2 Trust, but check the oracle first

If the oracle is wrong, every test agrees with a wrong answer. So `golden.c` checks itself against three cases whose answers are known by hand, and **exits before writing any file** if one disagrees:

- `identity`: W = I, so C must equal A.
- `all_ones`: every element of C must be 4.
- `cross_terms`: each output is a sum of two different products, compared against a hand-written table.

```
        if (C_identity[m*J + j] != (int32_t)A_identity[m*K + j]) {
          fprintf(stderr, "golden model self-check failed for 'identity'; no vectors
written\n");
```

```
    return 1;
}
```

3.3 Reproducible randomness

The 64 constrained-random cases come from a 3-line generator, not from `rand()`, whose sequence differs between C libraries:

```
static uint32_t xorshift32(uint32_t *s) {
  uint32_t x = *s;  x ^= x << 13;  x ^= x >> 17;  x ^= x << 5;  return *s = x;
}
```

Same seed (`0x5eed0006`) → same 64 cases, byte for byte, on any machine. The freeze report confirmed this by regenerating all 292 files in two separate clean directories and diffing them against the tracked copy: zero differences.

3.4 The files

File	Contents	Read with
`<case>_a.hex`	16 bytes, A row-major, zero-padded to 4×4	`$readmemh(..., A_case)`
`<case>_w.hex`	16 bytes, W (= B) row-major	`$readmemh(..., W_case)`
`<case>_c.hex`	M × 4 INT32 words, the expected C	`$readmemh(..., C_case)`
`<case>_meta.txt`	`M=4` `MODE=SIGNED` `K=4` `N=4`	`$fscanf`

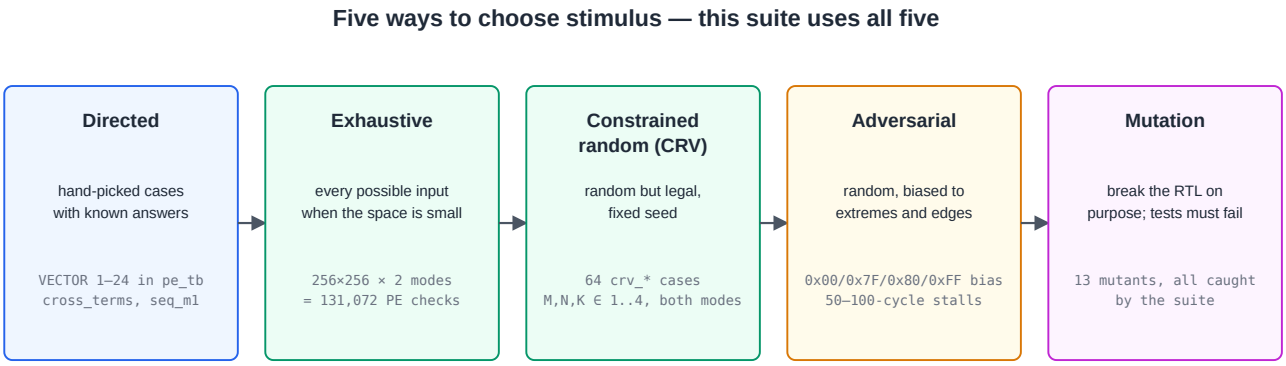
3.5 A subtle weakness, and why random cases exist

The three hand-checked cases share a property that is easy to miss: their weight matrices are all **symmetric**. For `cross_terms`,

```
W = | 1 0 2 0 |     W is equal to its own transpose, so  A × W  =  A × Wᵀ
    | 0 1 0 2 |
    | 2 0 1 0 |     For A = 1..16 both give  | 7 10  5  8 | ...
    | 0 2 0 1 |
```

A hardware bug that loads the weights transposed (PE(k, j) getting `W[j][k]`) would therefore pass all three. Only the random and CRV cases, whose weights are not symmetric, can catch it. Each kind of case covers what the others miss.

4. Choosing inputs: five strategies



Left to right: more inputs covered, less human effort per case. The last column tests the TESTS, not the design.

From hand-picked to deliberately broken

Directed tests are written for a reason: one specific corner. `unpu_pe_tb` vector 4 loads a new weight in the same cycle as a multiply, to prove the multiply uses the *old* weight.

Exhaustive tests are possible when the input space is small. A PE has 256 × 256 operand pairs per mode, 131 072 in total, which simulates in seconds. There is no reason to sample a space you can enumerate:

```
        for (w = 0; w < 256; w++) begin
          load_weight(w[7:0]);
          for (a = 0; a < 256; a++) begin
            act_in = a[7:0]; psum_in = 0; step();
            if (psum_out !== expected_mac(w[7:0], a[7:0], 0, mode)) errors++;
          end
        end
```

Constrained-random (CRV) tests cover spaces that are too big to enumerate. The constraints keep inputs legal (M, N, K in 1–4), and the fixed seed makes every failure reproducible.

Adversarial tests are random but skewed toward trouble. Uniform random bytes rarely hit `0x80` (−128) or `0x7F`, the values most likely to expose sign and overflow bugs, so the testbenches bias their draws:

```
        function automatic logic [7:0] biased_byte(ref logic [31:0] rng);   // from
unpu_pe_tb.sv
          logic [31:0] r1, r2;
          rng = xorshift32(rng); r1 = rng;
          if (r1[3:0] < 4'd2) begin                  // ~1 draw in 8 is a corner value
            rng = xorshift32(rng); r2 = rng;
            case (r2[1:0])
              2'd0: biased_byte = 8'h00;   2'd1: biased_byte = 8'hFF;
              2'd2: biased_byte = 8'h80;   default: biased_byte = 8'h7F;
            endcase
          end else
            biased_byte = r1[15:8];                  // otherwise an ordinary random byte
        endfunction
```

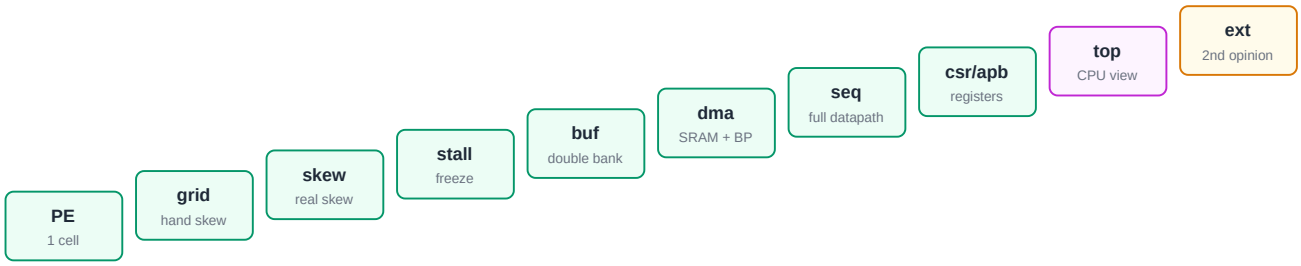
Plain uniform random would pick each of those four values only once in 256 draws, so the bias makes each of them roughly nine times more frequent (about 1 draw in 29 instead of 1 in 256).

Mutation testing is different in kind: it checks the tests, not the design (chapter 12).

5. Growing the device under test, one block at a time

The testbenches were written in the same order as the RTL, each adding one block to what was already proven:

Bottom-up build order: each step trusts only what the previous step proved



A failure at step n points at what step n added, because everything beneath it already passed on its own.

Each step trusts only what the step before proved

When a block is not yet included, **the testbench stands in for it**. `unpu_grid_tb` skews inputs by hand because `unpu_skew` is not in its DUT; `unpu_buf_tb` pulses `swap` itself because there is no sequencer yet. When `unpu_skew_tb` later replaces the hand skew with the real RTL and still passes, that proves the RTL matches the hand calculation.

Which RTL each testbench instantiates — the DUT boundary grows bottom-up

	pe	grid	skew	deskew	wbuf	actbuf	dma	seq	csr	apb	top
unpu_pe_tb	•										
unpu_grid_tb	•	•									
unpu_skew_tb	•	•	•	•							
unpu_stall_tb	•	•	•	•							
unpu_buf_tb	•	•	•	•	•	•					
unpu_dma_tb					•	•	•				
unpu_seq_tb	•	•	•	•	•	•	•	•			
unpu_csr_tb									•		
unpu_apb_tb									•	•	
unpu_top_tb	•	•	•	•	•	•	•	•	•	•	•
unpu_ext1_tb	•	•	•	•	•	•	•	•	•	•	•
unpu_ext2_tb	•	•	•	•	•	•	•	•	•	•	•

Modules not instantiated are replaced by the testbench itself: it forces weights, hand-skews inputs, or plays the sequencer.
Orange rows: testbenches written independently by a peer team and adapted (they never read model/vectors).

What each testbench instantiates

The payoff: when a test fails, the newest block is the prime suspect, because everything beneath it already passed on its own.

6. The compute core: PE, grid, skew, stall

6.1 `unpu_pe_tb`

Twenty-four directed vectors, the exhaustive sweep above, a randomized accumulator sweep (random `psum_in`, including values near `0x7FFF_FFFF`), randomized weight-load timing, and an adversarial sequence in which a **shadow model** of the PE runs alongside the DUT, cycle by cycle, with random freezes, resets and mode flips.

6.2 `unpu_grid_tb`: checking the timing contract

The grid test applies the contract's input timing by hand and checks its output timing:

```
        for (k = 0; k < 4; k++) begin                // A[m][k] enters row k at cycle m + k
          m = cyc - k;
          act_in[k] = (m >= 0 && m < case_M) ? A_case[m][k] : 8'h00;
        end
        step();                                       // finishes cycle 'cyc'
        for (j = 0; j < 4; j++) begin                 // C[m][j] valid at cycle m + j + 4 = cyc
+ 1
          m = cyc - j - 3;
          if (m >= 0 && m < case_M && psum_out[j] !== C_case[m][j]) errors++;
        end
```

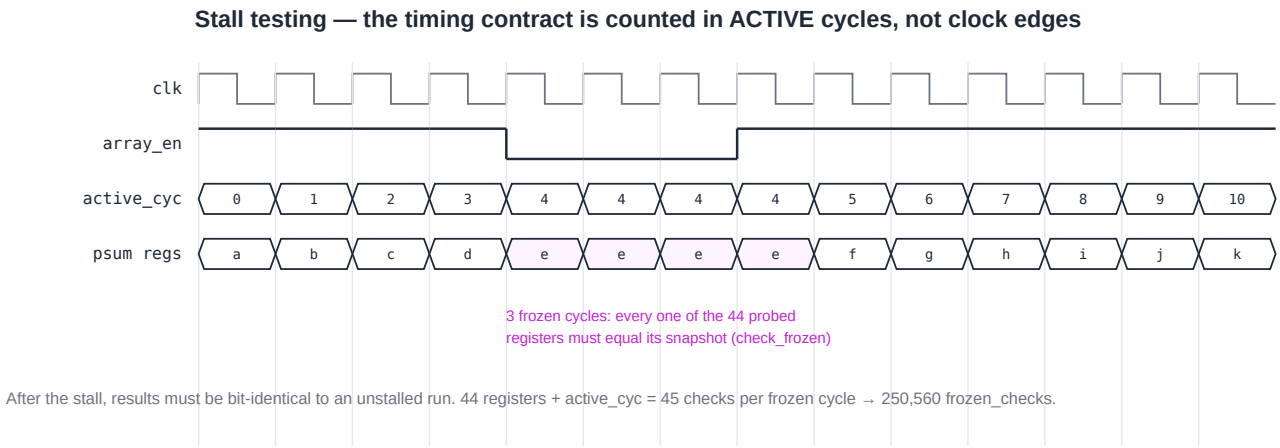
Two structural cases matter: a **walking-one** weight matrix, with a single 1 at each PE position in turn, proves each of the 16 PEs is wired to the right output; and **maximum-magnitude** inputs prove no partial sum is truncated.

6.3 `unpu_skew_tb`: no more hand skew

Raw rows go in, and realigned rows must come out at `m + 7`. Extra directed cases stress the **depth-0 lanes** (skew row 0, de-skew column 3), where adding a single flop shifts every result by one row.

6.4 `unpu_stall_tb`: freezing must be perfect

Dropping `array_en` must stop every datapath register at once. The testbench snapshots 44 internal registers, freezes the array for a random number of cycles, and checks every register on **every** frozen cycle:



Counting active cycles across a stall

```
        if (u_grid.g_row[2].g_col[1].pe.psum_out !== prev_pe_psum[2][1])
          begin errors++; $display("FAIL FREEZE [%s]: pe[2][1].psum_out changed", ctx); end
        else frozen_checks++;
```

This is **white-box** checking: hierarchical paths reach inside the DUT. It also uses a key idea: the timing contract is measured in *active* cycles (`active_cyc` only advances when `array_en` was 1), because a stall pauses the contract, not the clock. After resuming, C must be bit-identical to an unstalled run. 45 checks per frozen cycle add up to 250 560 `frozen_checks`.

7. The buffers: proving double-banking

The buffers exist so a load into one bank cannot disturb computation from the other. A test that loads, then computes, then loads again would pass even if there were only one bank. To prove the property, the load and the computation must **overlap**, so `unpu_buf_tb` runs them as two parallel processes:

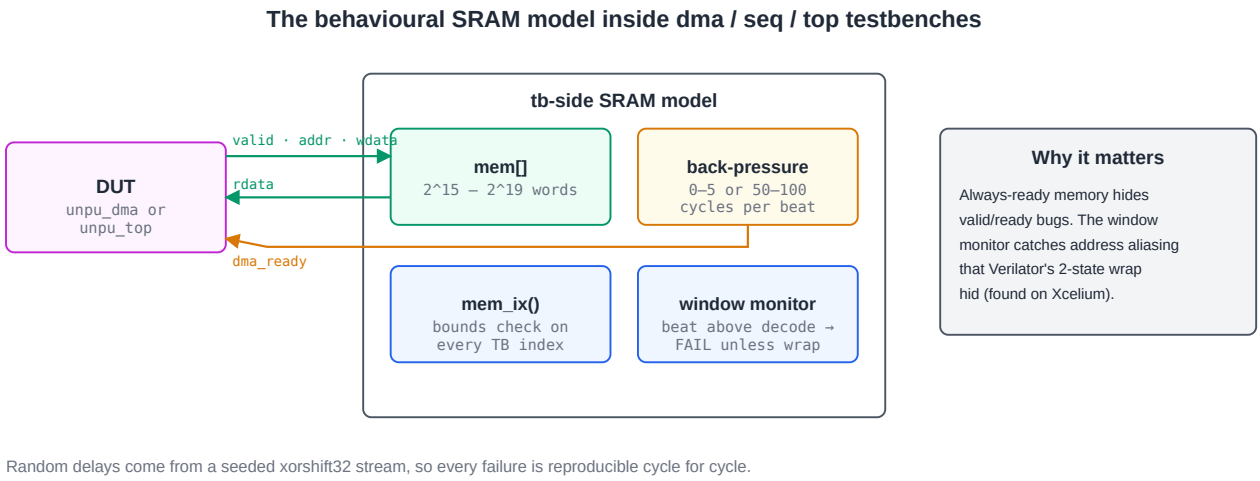
```
        fork
          run_compute_and_check("cross_terms (concurrent load in flight)", c_cur, 4, 4, 1'b0);
          begin
            repeat (2) step();                        // start the load inside the 11-cycle
compute
            drive_load_pair(a_tmp, w_tmp, 3, 2, 3);   // seq_mixed into the INACTIVE banks
          end
        join
        swap_both();                                  // only now does seq_mixed take effect
        run_compute_and_check("seq_mixed (post-concurrent-load swap)", c_cur, 3, 3, 1'b0);
```

The CRV phase chains all 64 cases so that case *i* loads while case *i − 1* computes. Further parts sweep every (K, N) masking boundary, check the weight buffer's reverse-row shifting for K = 1, 2, 3, and fire load/swap pairs back to back.

8. Memory and buses: the SRAM model and back-pressure

8.1 A memory that says “wait”

The DMA talks to SRAM through a valid/ready handshake. A memory that is always ready hides a whole family of bugs, for example an address that advances on `valid` instead of on `valid & ready`. So the testbenches’ SRAM model refuses each beat for a random 0–5 cycles (or 50–100 in extreme mode):



The behavioural SRAM model

```
        assign dma_rdata = mem[dma_addr[MEM_ADDR_BITS+1:2]];
        always_ff @(posedge clk)
          if (dma_valid && dma_ready && dma_wstrb == 4'hF) mem[dma_addr[MEM_ADDR_BITS+1:2]] <=
dma_wdata;
        // dma_ready is driven by a per-beat random delay drawn from xorshift32
```

8.2 Guarding the model itself

A testbench’s model can be wrong too. Two guards were added after a second simulator exposed a real problem (chapter 13):

- `mem_ix()` bounds-checks every index the testbench computes.
- A **window monitor** fails any DMA beat addressed above the modelled range, unless the test has declared deliberate wraparound:

```
        always @(posedge clk)
          if (dma_valid && dma_ready && (dma_addr >> (MEM_ADDR_BITS + 2)) != 0 &&
!wrap_expected)
            begin errors++; $display("FAIL [mem window]: DMA beat at addr 0x%08h ...",
dma_addr); end
```

8.3 `unpu_dma_tb`

Checks the exact address of every beat for every shape, write-back of only the real N columns at a 16-byte stride, wraparound near `0xFFFF_FFF0`, a 16-beat write under extreme stalls, and the 4-cycle `BUF_LOAD` staging.

8.4 Protocol checking

The adapted peer testbench `ext2` adds a **protocol checker**, a small always-on monitor that enforces a rule of the bus rather than a value:

```
        // a waiting request must not change
        always @(posedge clk)
          if (prev_valid_pending && (dma_addr !== prev_addr || dma_wdata !== prev_wdata ||
dma_wstrb !== prev_wstrb))
            begin $display("FAIL [PROTOCOL VIOLATION]: dma bus changed while valid&&!ready");
fail_count++; end
```

9. Control: sequencer, registers, APB

9.1 `unpu_seq_tb`

Puts the real sequencer in charge of the whole datapath and memory model. Beyond the 64 CRV cases run back to back without reset, it targets control behaviour:

- **Illegal configurations:** each dimension set to 0, 5, 6 and 7 must give ERROR with `error_code = 1` and no bus traffic, then a legal START must recover.
- **Stray starts:** `start` pulses injected at every stage of a running job must be ignored.
- **Bounded waiting**, so a hang becomes a failure instead of a frozen simulation:

```
        for (cyc = 0; cyc < 4000; cyc++) begin
          step();
          if (done) begin seen = 1; step(); break; end   // +1: DONE -> IDLE before the next
start
        end
        if (!seen) begin errors++; $display("FAIL [%s]: done never observed", label); end
```

9.2 `unpu_csr_tb`: a shadow model

Registers have no arithmetic to check, so the oracle is a **shadow model**: plain variables updated by the same rules as the RTL, compared after every cycle. The subtle part is ordering: START is registered one cycle after the write, and a new START clears DONE before a `done` pulse can set it.

```
        new_start_pulse = wen_v && (sel_v == 10'd6) && wdata_v[0];
        new_status_done = sh_start_pulse ? 1'b0 : (done_v ? 1'b1 : sh_status_done);
        step();
        check_eq1("mode_unsigned", mode_unsigned, ~sh_ctrl_signed);   // the SIGNED-bit
inversion
```

9.3 `unpu_apb_tb`

The slave is five lines of logic, so the test is about the protocol's edges. `pready == 1` is checked inside `step()`, so every cycle of the run checks it. The rule that matters most, that SETUP never commits a write, is tested with long SETUP phases, `pwrite` flipping between SETUP and ACCESS, and `penable` asserted without `psel`.

```
        psel = 1; penable = 0; step();   // SETUP: nothing may change
        penable = 1;           step();   // ACCESS: the write commits here
```

10. The whole macro, and a second opinion

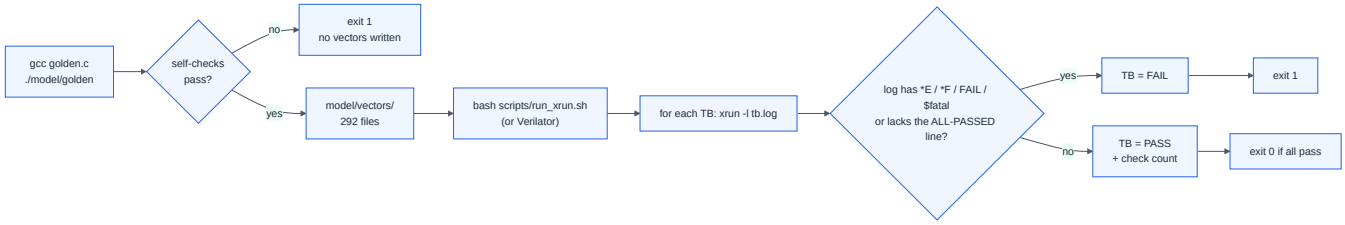
10.1 `unpu_top_tb`: black-box

Only the two external ports are used. The testbench behaves like firmware: APB writes to the registers, START, poll `npu_status`, then read C from the SRAM model. It also writes registers and polls status **during** a running job, and runs jobs with wraparound addresses. 343 341 checks.

10.2 `ext1` / `ext2`: independent tests

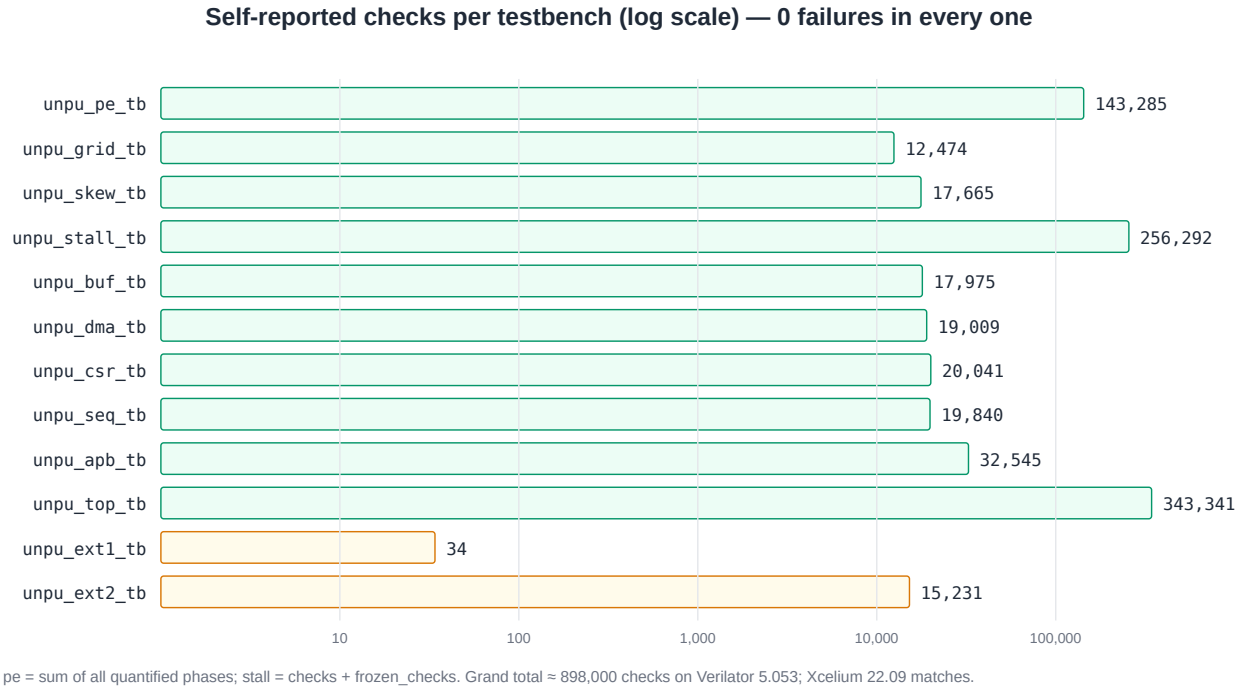
Everything so far shares one possible weakness: if the spec were misread once, `golden.c` and every testbench would agree with the same wrong reading. Two testbenches written by a peer team for a *different* systolic design were adapted to this one. They compute C with their own model from the operands in SRAM and never read `model/vectors`. Both pass, which is evidence that the spec reading is right, not just self-consistent.

11. Running a regression



From golden model to a PASS/FAIL table

`scripts/run_xrun.sh` compiles and runs each testbench and judges it **from its log**. xrun's exit code cannot be trusted for this: the testbenches end with `$finish`, which exits 0 even when checks failed. A testbench passes only if its log has none of `*E`, `*F`, `FAIL` or `$fatal`, **and** contains its own final ALL-PASSED line. A missing or empty log counts as a failure, so nothing fails silently.

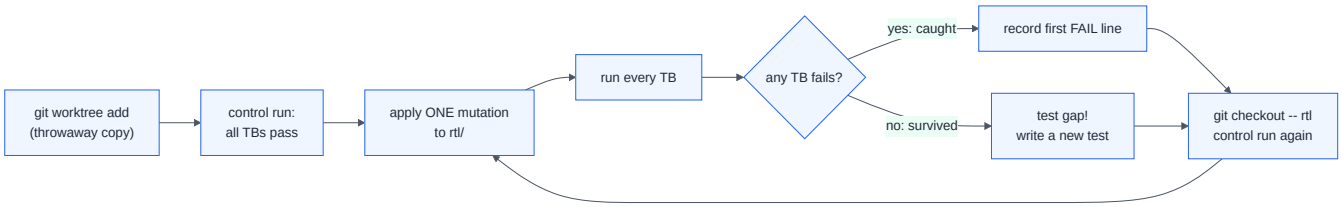


Checks per testbench, log scale

12. Testing the tests

12.1 Mutation testing

900 000 passing checks mean nothing if those checks could never fail. **Mutation testing** asks directly: break the design on purpose, and does the suite notice?



The mutation loop

#	Deliberate bug	Caught by
1	Sequencer leaves COMPUTE one cycle early	`seq` (6 537 failures), `top`, `ext1`, `ext2`
2	PE signed/unsigned branches swapped	`pe` (106 273), `grid`, `skew`, `stall`, `buf`, `seq`, `top`, `ext2`
3	Write-back stride 15 instead of 16 bytes	`dma` (1 774), `seq`, `top`, `ext1`, `ext2`
4a–c	One DMA address bit stuck at 0	`dma`, `seq`, `top`
5a–c	One high DMA address bit stuck at 1	window monitor, exact-address checks
6	One de-skew register ignores `array_en`	`stall` (names the exact register), `skew`
7a–c	Testbench SRAM shrunk back to its old size	bounds and window checks

All 13 were caught, usually by the earliest directed case. One honest gap: `ext1` alone misses mutant 2, because its data (1…16) means the same thing signed and unsigned. A test can only see a bug that its data can expose.

12.2 A second simulator

Verilator is fast and free but two-state and lenient. Running the same suite on Cadence Xcelium (four-state, strict IEEE 1800) found three **testbench** defects that Verilator had hidden (chapter 13). After the fixes, both simulators report the same counts on every testbench.

12.3 Random initial state

Real flip-flops power up with unknown values. Running `ext1`/`ext2` with Verilator's randomized initialization (`--x-initial unique`, six seeds) found that a monitor was looking at the bus before reset had finished. That was a testbench fault, fixed by gating the monitor with `rst_n`.

13. What verification found

Found by	What was wrong	Lesson
Xcelium	Variables used before their declaration in three TBs	Strict tools catch what lenient ones forgive
Xcelium	TB SRAM too small; Verilator **wrapped** the index so writes and reads aliased consistently, and the tests passed without really checking address separation	Bound-check models; add a window monitor
Xcelium	`$random(seed)` produced different stall lengths per simulator	Own the PRNG (`xorshift32`)
Random init	Monitor sampled before reset	Gate monitors with reset
Verilator	Open-ended `while (job_done !== 1)` hung the scheduler	Bounded loops with explicit caps
`seq_tb`	START issued during DONE is silently dropped	Wait one settle cycle
Review	All hand-checked W matrices symmetric	Keep non-symmetric random cases
Mutation #2	`ext1` data cannot tell signed from unsigned	Include values ≥ `0x80`

Every item is a testbench or methodology issue. **No RTL bug** was found by the campaign; the RTL has been unchanged since the APB revert. Mutation testing (chapter 12) shows that the suite was capable of finding RTL bugs.

14. Limits, and where to go next

This suite proves functional behaviour in simulation. It does not cover:

- **Timing.** Whether 50 MHz closes is a question for synthesis and STA, not RTL simulation.
- **Formal verification.** A formal tool proves a property for *all* input sequences, not only the ones simulated. The register file and the APB slave are small enough to be good first candidates.
- **Assertions (SVA).** Several checks here, like the protocol checker, are hand-written monitors. Written as SystemVerilog assertions (`assert property (@(posedge clk) dma_valid && !dma_ready |=> $stable(dma_addr));`), they could also run inside the full SoC simulation and in formal tools.
- **Coverage metrics.** The suite argues completeness through exhaustive sweeps, CRV and mutation testing. Functional coverage (`covergroup`) would *measure* which shapes, modes and stall positions were actually hit.
- **SoC-level firmware.** The course's RISC-V + C MNIST test needs the integrated SoC.

15. Appendix

Commands (from the repository root)

```
        gcc -std=c99 -Wall -Wextra -O2 model/golden.c -o model/golden && ./model/golden
        bash scripts/run_xrun.sh                  # ten frozen TBs on Xcelium
        bash scripts/run_xrun.sh ext1 ext2        # adapted peer TBs
        verilator --binary --timing -Wno-fatal --top-module unpu_seq_tb rtl/*.sv
tb/unpu_seq_tb.sv
        ./obj_dir/Vunpu_seq_tb
```

Testbenches

File	Lines	DUT	Checks
`unpu_pe_tb.sv`	934	pe	143 285
`unpu_grid_tb.sv`	618	grid	12 474
`unpu_skew_tb.sv`	778	skew + grid + deskew	17 665
`unpu_stall_tb.sv`	789	skew + grid + deskew	256 292
`unpu_buf_tb.sv`	820	wbuf + actbuf + compute	17 975
`unpu_dma_tb.sv`	1 167	dma + buffers	19 009
`unpu_seq_tb.sv`	1 310	seq + full datapath	19 840
`unpu_csr_tb.sv`	656	csr	20 041
`unpu_apb_tb.sv`	796	apb + csr	32 545
`unpu_top_tb.sv`	1 068	unpu_top	343 341
`unpu_ext1_tb.sv`	379	unpu_top	34
`unpu_ext2_tb.sv`	1 235	unpu_top	15 231